Supplementary Technical Report: Proxy-Guided Sampling for Approximate Graph Aggregation with Machine Learning Predicates

Anonymous Authors (Submission ID: 975)

This technical report serves as a supplementary document to our main submission. Due to strict space limitations in the conference format, we provide here:

1. A detailed real-world business motivation and case study (Section 1);
2. The end-to-end cost model and per-operation latency breakdown (Section 2);
3. The rigorous mathematical proof of uniform tree-homomorphism sampling (Section 3).

1 Real-World Motivation and Business Case Study

This section details a concrete, high-value business scenario demonstrating that Graph Aggregation with ML predicates (GA-ML) is an urgent requirement in modern AI-augmented databases, rather than an artificial extension of standard graph queries.

The Real-World Need for Multi-Modal ML on Graphs. Modern enterprises (e.g., Amazon, Alibaba) manage their data as heterogeneous property graphs (comprising User, Product, and Review vertices). However, crucial business insights are often locked inside unstructured attributes (images and text) attached to these vertices. For instance, a quality-control analyst investigating user dissatisfaction needs to query: *"Find all transactions where a user purchased a wooden furniture product and left an angry/negative review."*

Traditional relational filters cannot express this. It strictly requires multi-modal ML predicates: a Computer Vision (CV) model to verify "wooden furniture" from product images, and an NLP sentiment model to detect "angry" emotions in review texts.

The Business Value of Graph Aggregation (COUNT/SUM/AVG). In enterprise scenarios, merely retrieving a massive list of matched subgraphs is unhelpful for decision-making. Analysts fundamentally need macro-level aggregations over these topological structures. As detailed in our Introduction, applying an aggregation function λ provides distinct business KPIs:

- $\lambda = \mathsf{COUNT}$: Quantifies the total transaction volume of these controversial orders.
- $\lambda = \mathsf{SUM}$: Calculates the total sales revenue at risk (by aggregating the `price` attribute of the matched subgraphs).
- $\lambda = \mathsf{AVG}$: Identifies the average unit price of the criticized items, helping to pinpoint if the issue lies in high-end or low-end markets.

2 End-to-End Cost Model and Latency Analysis

The total execution cost of a GA-ML query in PROXY can be rigorously decoupled into Offline Setup Cost (C_{offline}) and Online Query Cost (C_{online}).

2.1 Offline Setup Cost (Amortized to Zero)

The offline cost comprises model initialization and optional fine-tuning:

- **Zero-Shot Plug-and-Play**: PROXY natively supports off-the-shelf pre-trained models from Hugging Face (e.g., Oracle model: `microsoft/deberta-v2-xxlarge-mnli`, Proxy model: `sileod/deberta-v3-base-mnli-fever-anli`). For these models, C_{offline} strictly equals the one-time memory/GPU loading overhead.
- **Task-Specific Fine-Tuning (Optional)**: To systematically evaluate the impact of proxy model degradation (e.g., $F_1 \in [0.34, 0.89]$), we lightly fine-tuned base backbones (e.g., `microsoft/deberta-v3-base`) using a small, disjoint background split (500–1000 instances). This offline process takes only a few minutes and strictly amortizes to zero during online query execution.

2.2 Online Query Cost and Per-Operation Latency

The online latency C_{online} determines the practical interactivity of the system. It consists of graph structural preparation, proxy scoring, and oracle verification:

$$C_{\text{online}} = \underbrace{C_{\text{CS}} + \left(K \cdot c_{\text{tree}}\right)}_{\text{In-Memory Graph Operations}} + \underbrace{\left(|\hat{\Psi}| \cdot \bar{c}_{\text{proxy}}\right) + \left(m \cdot \bar{c}_{\text{eff}}\right)}_{\text{GPU ML Inference}} \tag{1}$$

To illustrate the massive order-of-magnitude differences among these operations, we benchmark the per-operation costs using a typical Natural Language Inference (NLI) query configuration (Proxy: `sileod/deberta-v3-base-mnli-fever-anli` vs. Oracle: `microsoft/deberta-v2-xxlarge-mnli`) on a single NVIDIA RTX 3090 GPU):

1. $C_{\mathbf{CS}} + K \cdot c_{\mathbf{tree}}$: Time required to construct the Candidate Space (CS) summary and draw K tree-homomorphism samples. Executed serially on the CPU, building the CS and drawing $K = 60{,}000$ samples takes $\approx$ 1000 ms. The amortized cost per sample is merely $\sim$ 16 μ**s (microseconds)**.
2. $|\hat{\Psi}| \cdot \bar{c}_{\mathbf{proxy}}$: Cost of evaluating the lightweight proxy model across all distinct semantic projections. Through batched GPU inference, c_{proxy} operates strictly in the **millisecond** regime ($\approx$ 1.8 ms per item, achieving an inference throughput of 550–960 items/s).
3. $m \cdot \bar{c}_{\mathbf{eff}}$: Cost of executing the heavy Oracle model over m sampled projections. This represents the absolute performance bottleneck ($>$ 95% of total latency), as the 1.5B-parameter oracle processes only $\approx$ 13 items/s, taking $\sim$ 78 **ms (milliseconds)** per item.

As defined in the main paper, $\bar{c}_{\text{eff}} = \sum_{i=1}^{k} \rho_i r_i c_i$ encapsulates both the *short-circuiting* probability (ρ_i) and the *cache miss rate* ($r_i = U_i/|\hat{\Psi}|$). Because $\bar{c}_{\text{eff}} \gg \bar{c}_{\text{proxy}}$ (e.g., 78 ms vs. 1.8 ms), strictly bounding the oracle sample size m via PROXY's pilot-free budget allocation is mathematically and practically indispensable to rapidly break even against exact evaluation.

3 Proof of Uniform Tree-Homomorphism Sampling

In Section IV-C of the main paper, we proposed a top-down sampling procedure to draw a tree homomorphism $h \in \Omega$ over the Candidate Space (CS) summary. Below, we formally prove that this procedure ensures every valid homomorphism in Ω is sampled uniformly at random with exact probability $1/|\Omega|$.

Sampling Procedure Recap. Let T be a spanning tree of the query graph Q with root r. The number of homomorphisms of the subtree of T rooted at u, given u is mapped to v, is recursively calculated via dynamic programming:

$$H(u,v) = \begin{cases} 1 & \text{if } u \text{ is a leaf in } T \\ \prod_{u' \in \text{ch}(u)} \sum_{v' \in C(u' | u \to v)} H(u', v') & \text{if } u \text{ is an internal node} \end{cases} \tag{2}$$

The total number of homomorphisms is $|\Omega| = \sum_{v \in C(r)} H(r, v)$.

Our sampling procedure draws $h(r) = v_r \in C(r)$ with probability $P(r \to v_r) = \frac{H(r,v_r)}{|\Omega|}$. Then, iteratively for each mapped query vertex $u \to v$ and each of its unmapped children $u' \in \text{ch}(u)$, it selects a candidate data vertex $v' \in C(u' \mid u \to v)$ with probability proportional to $H(u', v')$:

$$P(u' \to v' \mid u \to v) = \frac{H(u', v')}{\sum_{v'' \in C(u' | u \to v)} H(u', v'')} \tag{3}$$

Theorem 1 (Uniform Tree Sampling)**.** *For any complete valid tree homomorphism $h \in \Omega$, the probability of generating h using the proposed top-down sampling procedure is exactly $P(h) = \frac{1}{|\Omega|}$.*

Proof. Let T_u denote the subtree rooted at node u. We first prove by structural induction (bottom-up) that the conditional probability of sampling the exact partial homomorphism $h|_{T_u}$ for the subtree T_u, given that its root u is already mapped to $h(u)$, is exactly $\frac{1}{H(u,h(u))}$. Let this conditional probability be denoted as $P(T_u \text{ matches } h \mid u \to h(u))$.

Base Case: If u is a leaf in T, there are no children to map. The subtree T_u consists only of u. Given $u \to h(u)$, the mapping is trivially complete with probability 1. By definition, $H(u, h(u)) = 1$. Thus, $P(T_u \text{ matches } h \mid u \to h(u)) = 1 = \frac{1}{1} = \frac{1}{H(u,h(u))}$. The base case holds.

Inductive Step: Assume u is an internal node, and the property holds for all of its children $u' \in \text{ch}(u)$. The probability of generating the exact subtree mapping $h|_{T_u}$ given $u \to h(u)$ is the product of the probabilities of independently mapping each child $u' \to h(u')$ and recursively matching their respective subtrees $T_{u'}$.

$$P(T_u \text{ matches } h \mid u \to h(u)) = \prod_{u' \in \text{ch}(u)} \Big[P(u' \to h(u') \mid u \to h(u)) \times P(T_{u'} \text{ matches } h \mid u' \to h(u')) \Big]$$

By substituting the sampling probability $P(u' \to h(u') \mid u \to h(u))$ and applying the inductive hypothesis $P(T_{u'} \text{ matches } h \mid u' \to h(u')) = \frac{1}{H(u', h(u'))}$, we obtain:

$$\begin{aligned} P(T_u \text{ matches } h \mid u \to h(u)) &= \prod_{u' \in \text{ch}(u)} \left[\frac{H(u', h(u'))}{\sum_{v'' \in C(u' | u \to h(u))} H(u', v'')} \times \frac{1}{H(u', h(u'))} \right] \\ &= \prod_{u' \in \text{ch}(u)} \frac{1}{\sum_{v'' \in C(u' | u \to h(u))} H(u', v'')} \\ &= \frac{1}{\prod_{u' \in \text{ch}(u)} \sum_{v'' \in C(u' | u \to h(u))} H(u', v'')} \end{aligned}$$

By the recursive definition of $H(u, v)$ from Eq. 2, the denominator is precisely $H(u, h(u))$. Therefore:

$$P(T_u \text{ matches } h \mid u \to h(u)) = \frac{1}{H(u, h(u))}$$

This completes the induction.

Final Probability Calculation: The entire homomorphism h is formed by first mapping the root r to $h(r)$ and then recursively mapping the subtree T_r. The total probability $P(h)$ is:

$$\begin{aligned} P(h) &= P(r \to h(r)) \times P(T_r \text{ matches } h \mid r \to h(r)) \\ &= \frac{H(r, h(r))}{|\Omega|} \times \frac{1}{H(r, h(r))} \\ &= \frac{1}{|\Omega|} \end{aligned}$$

Thus, every valid homomorphism $h \in \Omega$ is sampled with equal probability $\frac{1}{|\Omega|}$. This rigorous bottom-up structural induction completes the proof of uniform sampling. □